Verification, Not Self-Talk, Makes Small Models Big

No system built today turns a 20B-parameter model into a trillion-parameter generalist. Knowledge storage is bounded at roughly 2 bits per parameter, and the hardest reasoning problems still favor scale. What the evidence does support, and strongly, is a narrower claim: **on task families with a sound external check (code with tests, exact-answer math, formal proofs) and on factual lookup backed by retrieval, a small model inside the right system matches models 10-100x larger at lower cost per solved problem.** The headline cases come from that regime. A 1B model beats a 405B model on MATH with a process reward model. An 8B prover beats a 671B prover on miniF2F with a proof checker in the loop. An 11B retrieval model beats 540B PaLM on Natural Questions. gpt-oss-20b at high reasoning effort with tools scores 98.7 on AIME 2025, against 92.5 for gpt-oss-120b without them. Gains vanish or reverse when the system only re-prompts the same model. Intrinsic self-correction, LLM-as-judge checking, multi-agent debate, persona prompts, planners on one-shot tasks and query-text routers on reasoning workloads all add tokens and frequently flip correct answers to wrong ones. Neo's measured results fit this second pattern almost exactly: ties at 3-8x tokens on QA, and a drop from 95% to 82-88% on code at 5.5 calls per problem. The proposal below rebuilds Neo as a **verify-first cascade**: a cheap first attempt, checks grounded in execution and retrieval, escalation only on objective failure, and a flywheel that distills verified 120b solutions back into 20b. It is framed around a falsifiable claim. On post-cutoff coding and exact-answer math, gpt-oss-20b plus the system matches raw gpt-oss-120b at high effort within a pre-registered margin, at lower cost per solved problem. On long-tail factual QA, 20b plus gated retrieval beats 120b closed-book.

Capability splits three ways, and only some of it compresses

"Smarter" decomposes into three things that respond to different levers, and most of the confusion in this field comes from treating them as one. The first is **stored knowledge**, which is capacity-bound: language models hold about 2 bits of knowledge per parameter, even when quantized to int8 (Allen-Zhu & Li). The second is **procedural reasoning skill**, which compresses well through distillation and test-time search. The third is **exact computation** (arithmetic, symbolic algebra, execution, proof checking), which no model needs to internalize if a tool does it.

gpt-oss's own model card shows the split clearly. Between 20b and 120b, MMLU differs by only 4.7 points at high effort (85.3 vs 90.0). SimpleQA, a long-tail factual benchmark, differs by 2.5x (6.7% vs 16.8%), and the 20b hallucinates on 91.4% of those questions (gpt-oss model card). On competition math the two are nearly tied at high effort (AIME 2025: 91.7 vs 92.5). The small model's deficit is knowledge first and hard-reasoning second. That points to retrieval for the first problem and verified search for the second.

Model releases make the same point from the training side. The "Densing Law" fit finds capability per parameter doubling roughly every 3.3 months on the benchmarks studied (arXiv 2412.04315). Qwen3-4B (April 2025) matched R1-Distill-Qwen-32B (January 2025) on AIME'24 at one-eighth the size (Qwen3 report). Qwen3.5-9B now beats gpt-oss-120b on GPQA-Diamond (81.7 vs 80.1) and MMLU-Pro, but trails it by 17 points on LiveCodeBench v6 (65.6 vs 82.7) (Qwen3.5-9B card). Phi-4 beats GPT-4o on GPQA and MATH yet scores 3.0 on SimpleQA against GPT-4o's 39.4 (Phi-4 report). Density gains concentrate in math and STEM reasoning. Coding, agentic breadth and factual recall lag.

A "trillion-parameter-class" ambition is therefore achievable only per task family, never across the board. The table summarizes what each lever closes and what it costs, based on the evidence in the sections that follow.

Lever	What it closes	Strongest evidence	Cost profile	Solo-feasible around gpt-oss
Verified test-time compute (sampling + execution/PRM/proof check)	Reasoning gap where small model's pass@k is high	7B MATH 58.8→90.0 with MCTS+PRM; SWE-bench Lite 15.9→56% at 250 samples	Linear in samples; optimum often <10 with imperfect tests	Yes

Lever	What it closes	Strongest evidence	Cost profile	Solo-feasible around gpt-oss
Reasoning-effort budget	Reasoning depth	20b AIME'25 37.1 (low) → 91.7 (high)	Tens of thousands of CoT tokens per hard problem	Yes, one parameter
Tools (code, calculator, solver)	Exact computation	20b AIME'25 +20 pts with tools at low effort; GPQA only +1-3	Cheap per call	Yes, already in Neo
Retrieval	Long-tail facts	Atlas-11B beats PaLM-540B on NQ; MMLU only +3-5	Index + retriever; noise risk	Yes
Distillation / on-policy distillation	Procedural skill, permanently	Qwen3-8B on-policy distill beats RL at ~1/10 GPU-hours	Teacher traces + LoRA/full FT	Yes for LoRA SFT; on-policy needs teacher logprobs
RL with verifiable rewards	Pass@1 sharpening	DeepScaleR-1.5B AIME 28.8→43.1	Thousands of GPU-hours; rarely expands pass@k	Marginal
Efficient architectures (MoE, hybrids, looping, quantization)	Mostly cost, not capability	Nemotron 3 Super ~2.2x gpt-oss-120b throughput at "comparable accuracy"	Requires pretraining to change	Only by model swap
Routers / cascades	Cost on easy traffic	LLMRouterBench: best routers save ≤31.7% at best-single accuracy	Router + eval set	Yes, if gated on verification
Self-critique, debate, personas, planners	Nothing measurable on single-step tasks	Debate loses to self-consistency at matched calls	2-50x tokens	Remove

Test-time compute pays only when a sound verifier picks the winner

The case for "small model plus inference beats big model" is real but conditional. Snell et al. show that allocating test-time compute per prompt by difficulty is **more than 4x more efficient than best-of-N**. In FLOP-matched comparisons a smaller model beats a 14x larger one, but only "on problems where a smaller base model attains somewhat non-trivial success rates". On the hardest problems, extra pretraining wins (arXiv 2408.03314). Liu et al. push the result to extremes, with a 1B model beating 405B on MATH-500 and a 3B beating 405B on both MATH-500 and AIME24, but every such case uses a strong, task-specific process reward model on short checkable answers (arXiv 2502.06703). rStar-Math's MCTS with a self-evolved process preference model lifts Qwen2.5-Math-7B from **58.8% to 90.0% on MATH** and Phi3-mini-3.8B from 41.4% to 86.4% (arXiv 2501.04519). None of these headlines transfers automatically to open-ended code or agent tasks.

The reason is the gap between *coverage* and *selection*. Large Language Monkeys found that coverage (any sample correct) grows log-linearly with sample count across four orders of magnitude. With an automatic verifier, coverage converts directly into accuracy: DeepSeek-Coder-V2 on SWE-bench Lite rose from **15.9% at one sample to 56% at 250**, against a single-attempt state of the art of 43%. Without a verifier, majority voting and reward models plateau after a few hundred samples (arXiv 2407.21787). Stroebl et al. add the hard ceiling: an imperfect verifier's false-positive rate caps resample-until-pass accuracy. Weaker models generate more test-passing wrong programs, so no amount of resampling lets a weak model match a sufficiently stronger model's single-sample accuracy. The optimal sample count is **often under 10** (arXiv 2411.17501).

For code, execution is the verifier that works. CodeT ranks samples by agreement with model-generated tests and lifts code-davinci-002 HumanEval pass@1 from about 47.0% to **65.8%** (arXiv 2207.10397). LEVER's learned verifier over execution results adds 4.6-10.9 points across four benchmarks (arXiv 2302.08468). AlphaCode reached roughly median-competitor Codeforces level by filtering massive sample sets on public tests and clustering survivors by behavior (arXiv 2203.07814). Self-Debugging gains up to 12% when unit-test feedback exists, but only 2-3% on Spider, which has none (arXiv 2304.05128). Small models also make poor verifiers on calculation and fact-checking. Routing candidates through a code interpreter before the small-model check fixes this, and lets Llama-3.2-1B beat Llama-3.1-8B on MATH (T1, arXiv 2504.04718).

Verification by the model's own judgment fails. Huang et al. found intrinsic self-correction lowers accuracy. GPT-4 falls from **95.5% to 89.0% on GSM8K** after two self-correction rounds, and correct-to-incorrect flips outnumber the reverse. Earlier positive results relied on oracle labels to decide when to stop (arXiv 2310.01798). Kamoi et al.'s survey finds no successful self-correction from prompted feedback outside tasks exceptionally suited to it (arXiv 2406.01297). Olausson et al. report that code self-repair gains are "often modest" and "sometimes not present at all" once cost is counted. The bottleneck is the quality of the model's feedback on its own code (arXiv 2306.09896). Under a fixed budget, plain self-consistency beats generative-reward-model verification at most practical budgets; GenRM needs up to 8x the compute just to match it (arXiv 2504.01005). More calls can also make results worse. Chen et al. show vote-based systems improve and then degrade as calls grow, because extra calls help easy queries and hurt hard ones (arXiv 2403.02419).

Neo's 95%→82-88% coding regression is the textbook version of this failure. At a 95% base rate there are only two problems in forty to gain, and every unverified rewrite risks breaking a correct answer. An LLM checker with a 10-15% false-negative rate, feeding a refine step that "fixes" flagged solutions, is enough to explain the drop. That diagnosis is an inference, and Neo can confirm it by logging flip directions.

The cheapest test-time lever is already inside the model. gpt-oss exposes reasoning effort, and its effect dwarfs most scaffolds: 20b moves from 37.1 to 91.7 on AIME 2025 and from 37.4 to 60.7 on SWE-bench Verified between low and high. **20b at high effort roughly matches 120b at medium** on AIME and SWE-bench, and reaches Codeforces Elo 2516 with tools against 120b's 2463 without (gpt-oss model card). The knob is not monotone everywhere. Anthropic-affiliated work documents inverse scaling, where longer reasoning hurts on distractor-heavy counting, spurious-feature regression and long constraint-tracking (arXiv 2507.14417). The Holistic Agent Leaderboard likewise found higher reasoning effort reduced accuracy in most of its agent runs (HAL, arXiv 2510.11977). Effort should therefore be set per task family by measurement, not by maximizing it.

Retrieval and tools buy 10-50x on facts and arithmetic but single digits on exams

Externalizing knowledge works where the answer is a retrievable fact. RETRO matched GPT-3 and Jurassic-1 on the Pile with 25x fewer parameters, though part of that gain came from train/test overlap with its retrieval database (arXiv 2112.04426). Atlas-11B scored **42.4 on Natural Questions 64-shot versus PaLM-540B's 39.6**, and 84.7 vs 81.4 on TriviaQA. Swapping a 2017 Wikipedia index for a 2020 one lifted accuracy on 2020 questions from 1.5 to 53.1 with no retraining (Atlas). MassiveDS showed a 7B model with a sub-100B-token datastore beating closed-book Llama-2 13B and Llama-3 8B on TriviaQA and NQ, with retrieval-based LMs compute-optimal at equal training compute (arXiv 2407.12854).

The same studies show sharp limits on exam-style tasks. MassiveDS gave Llama-2 7B **+12.9 on TriviaQA but only +3.5 on MMLU** and +2.8 on MedQA (MassiveDS). Atlas reached only 47.9 on 5-shot MMLU, about 20 points behind closed-book Chinchilla-70B. Its larger +12 MMLU gain required jointly training retriever and reader on MMLU-style data (Atlas). Domain-matched corpora are the exception that proves the rule: MedRAG raised six LLMs by up to 18% and brought GPT-3.5 and Mixtral to GPT-4 level on medical QA (arXiv 2402.13178). Parametric alternatives such as Meta's memory layers (arXiv 2412.09764) and DeepSeek's Engram show similar magnitudes, roughly +3-4 MMLU at 8-27B, but they are pretraining-time changes that cannot be retrofitted onto gpt-oss. For Neo, Wikipedia search is the wrong corpus for MMLU's professional subjects, and MMLU is the wrong benchmark for measuring retrieval. SimpleQA, where 20b sits at 6.7%, is where tens of points are available.

Retrieval also has a failure mode that mirrors self-correction. ClashEval found top models, GPT-4o included, adopted incorrect retrieved content over their own correct prior **more than 60% of the time**, and less confident models adopted it more readily (arXiv 2404.10198). On-topic passages that lack the answer hurt more than random ones (arXiv 2401.14887), and facts in the middle of long contexts get underused (arXiv

2307.03172). Always-on retrieval is therefore a liability. Retrieval should be gated on low closed-book agreement, reranked, and placed at the start or end of the context.

Tools produce the largest multiples in the literature because they remove exact computation from the model entirely. PAL's program execution beat PaLM-540B chain-of-thought on GSM8K by 15 points (arXiv 2211.10435). Goedel-Prover-V2-8B reaches **84.6% on miniF2F at pass@32 versus 82.4% for the 671B DeepSeek-Prover-V2**, roughly 80x smaller, because Lean checks every proof (arXiv 2508.03613). AlphaGeometry's symbolic engine plus a small LM solved 25 of 30 olympiad geometry problems (Nature). For modern reasoning models the tool dividend shrinks as effort rises. gpt-oss-20b gains about 20 points on AIME 2025 from tools at low effort, 18 at medium and 7 at high, and only 1-3 points on GPQA at any effort (gpt-oss model card). Neo's calculator with exact rationals, `solve()` and `run_python` are the right investments. They should be used as *verifiers* of the model's answers (program-aided cross-checks) as well as solvers, and nobody should expect them to move MMLU.

Distillation injects capability; architectures mostly cut the bill

On the training side, distillation from a stronger teacher is the best-documented way to make a small model genuinely smarter. DeepSeek's SFT-only distills, trained on 800k R1 samples, put R1-Distill-Qwen-14B at **69.7 on AIME'24 versus GPT-4o's 9.3**. DeepSeek states that distilling a larger model's reasoning patterns beats letting small models discover them through RL (R1-Distill card). Qwen3's controlled comparison on an 8B model is the cleanest cost evidence. Starting from the same checkpoint, RL reached AIME'24 67.6 in 17,920 GPU-hours with pass@64 unchanged. On-policy distillation reached **74.4 in 1,800 GPU-hours** and raised pass@64 to 93.3 (Qwen3 report).

Yue et al. (NeurIPS 2025 oral) explain the pattern. Standard RL with verifiable rewards sharpens sampling toward paths the base model could already find, winning at small k, while base models match or beat it at large k. Distillation, by contrast, "can introduce new reasoning patterns" (arXiv 2504.13837). ProRL argues prolonged, diverse RL can expand the boundary, chiefly on synthetic puzzles at 1.5B (arXiv 2505.24864). Curation beats volume. LIMO's 817 examples reach 63.3% on AIME24, while uncurated NuminaMath-100k *degraded* a base from 49.9% to 32.3% (arXiv 2502.03387). For narrow tasks, LoRA Land's 310 fine-tunes averaged +10 points over GPT-4 across 31 tasks, though GPT-4 stayed ahead on broad tasks like coding and MMLU, and the benchmark was vendor-run (arXiv 2405.00732).

Two cautions apply. Contamination inflates many small-model results: R1-Distill-Qwen-32B drops from 72.6 on AIME'24 to 49.6 on the fresher AIME'25 (Qwen3 report). Qwen2.5's "RL works even with random rewards" gains collapse on post-release benchmarks (arXiv 2507.10532). And synthetic data must accumulate alongside real data rather than replace it, or models collapse (arXiv 2404.01413).

Architectural innovations mostly lower cost rather than raise capability, and almost all require pretraining a solo developer cannot do. **MoE is the established capability-per-FLOP win** (Abnar et al., ICML 2025), and gpt-oss already uses it: 120b runs 5.1B active of 117B total parameters, 20b runs 3.6B active of about 21B (HF blog).

That fact has an uncomfortable implication for Neo. **Per token, 120b costs only about 1.4x the active compute of 20b**, so a 20b→120b cascade has limited cost headroom. The bigger differences are memory (about 5.6x in total parameters) and whatever price gap the user's endpoints charge.

Looped models such as Ouro trade FLOPs for parameters, giving about 2-3x parameter efficiency on reasoning with unchanged knowledge storage (arXiv 2510.25741). Hybrid SSM/attention models deliver throughput and KV-cache savings at long context. Nemotron 3 Super claims up to 2.2x gpt-oss-120b throughput at "comparable accuracy" (arXiv 2604.12374), but an independent single-GPU test of Qwen3.5-9B found only +16-37% speedups at 32K (sotaaz). At a fixed memory budget, about 4-bit quantization of a larger model beats a smaller model at higher precision (arXiv 2212.09720), and gpt-oss already ships at MXFP4.

The tiny recursive models (TRM at 7M parameters, 44.6% on ARC-AGI-1) are real but narrow, trained largely at test time on the task distribution (ARC Prize 2025). ARC Prize's ablation of HRM found the outer refinement loop, not the "hierarchical" architecture, drove results (ARC Prize). The same report shows refinement loops around big models moved the 2025 frontier further: Poetiq's loop took Gemini 3 Pro from 31% to **54% on ARC-AGI-2 at about $30 per task** (ARC Prize 2025). For Neo, the actionable architecture moves are model swaps (evaluate denser small models as the cascade's first tier) and, if self-hosting, speculative decoding with 20b drafting for 120b, since they share a tokenizer (Leviathan et al.).

Routers and scaffolds earn their cost only by injecting grounded information

Early routing papers promised dramatic savings: up to 98% for FrugalGPT (arXiv 2305.05176), over 2x for RouteLLM (arXiv 2406.18665) and over 50% for AutoMix (NeurIPS 2024). The January 2026 LLMRouterBench re-evaluation, covering 33 models, 21 datasets and about 400K instances, is sobering. The best routers gained at most about 4% accuracy over the best single model and **saved at most 31.7% cost at matched accuracy**. Binary strong/weak routers "struggled" to save anything without losing accuracy, and OpenRouter's router scored 24.7% below best-single (arXiv 2601.07206). Savings also shrink on reasoning. RouteLLM's cost reduction falls from 3.66x on MT-Bench to 1.4-1.5x on MMLU and GSM8K (RouteLLM). Query-text routing is weakest exactly where Neo operates. Post-hoc signals that inspect the cheap model's actual answer, test results above all, are the strongest.

Agent-scaffold evidence draws the same line. Kapoor et al. found simple baselines matched elaborate HumanEval agents. GPT-4 "warming" scored 93.2% for $2.45, while LATS scored 88.0% for **$134.50**. A cheap-first escalation baseline was the lowest-cost point at 85.0% for $0.27 (arXiv 2407.01502). Multi-agent debate generally fails to beat self-consistency at matched compute (arXiv 2502.08788). Mixing models in Mixture-of-Agents loses to sampling the single best model several times (Self-MoA: +6.6% on AlpacaEval 2.0) (arXiv 2502.00674). Personas across 162 roles produced no accuracy gain over a no-persona control (arXiv 2311.10054). Multi-agent systems fail mostly through system design and weak task verification (MAST, arXiv 2503.13657). Scaffolds clearly help when they give the model an environment: a roughly 100-line bash-only agent reaches 65-74% on SWE-bench Verified (mini-SWE-agent).

Read against this literature, Neo's results are what its design predicts, not a tuning failure. Its planner, LLM checker, self-refine ("pursue"), persona and mood prompts are all self-talk: they re-prompt the same model without new information. They tie on GSM8K/ARC/MMLU at 3-8x tokens, a Pareto loss, and lose points on code, where they overwrite correct drafts. On hard LiveCodeBench, Snell's caveat applies. When 20b's pass probability is near zero, rounds of 20b planning and refinement cannot recover the problem. They burn budget and can anchor the eventual 120b call on a wrong plan.

The measurements themselves need tightening too. At 30 questions, a 95% confidence interval on an 80% score spans roughly ±14 points, so 80 vs 77 on MMLU is noise. Neo's raw MMLU of 77% also sits below the model card's 80.4 even at *low* effort, which suggests answer extraction or effort settings are costing more than any tool adds (gpt-oss model card).

A verify-first cascade gives Neo a claim it can actually prove

The claim to test

The proposal reframes "as smart as a trillion-parameter model" into three nested, falsifiable claims, ordered by confidence.

Claim A (primary) is non-inferiority on verifiable families. On post-August-2025 LiveCodeBench problems and on fresh exact-answer math (AIME 2026, HMMT), Neo-V running gpt-oss-20b first matches raw gpt-oss-120b at high effort. "Matches" means paired pass@1 within a pre-registered margin (for example 2 points). Neo-V must also show strictly lower cost per solved problem, measured in dollars on the user's actual endpoints and

in total tokens. The model card's 20b-with-tools results make this plausible: Codeforces Elo 2516 vs 120b's 2463 without tools.

Claim B is that 20b plus gated retrieval beats 120b closed-book on long-tail factual QA (SimpleQA 6.7% vs 16.8%) at lower cost. **Claim C (stretch)** applies the same system with 120b as the base model and tests whether it matches a frontier closed model on one chosen task family at lower cost per solved problem. That is the honest form of the "far less compute" ambition.

Claims stay scoped to task families where a check exists. Neo-V should explicitly *not* claim parity on knowledge-heavy exams or open-ended generation.

The architecture

``` task – FRONT DOOR: classify by verifiability – code with examples – Tier 0 check: sandbox run on task examples + CodeT-style generated tests – exact-answer math – Tier 0 check: CoT answer vs PAL program (run_python, rationals, solve()) – factual – closed-book k samples; retrieve + rerank only if agreement is low └─ open-ended – single call or small-k self-consistency; no critique loop

TIER 0 gpt-oss-20b, 1 sample, effort tuned per family ── passes check? ── return | fails TIER 1 gpt-oss-20b, k ≤ 4-8 samples, high effort; select by execution agreement / vote; one repair fed with real failing-test output; "do-no-harm" acceptance ── passes? ── return | fails, or difficulty gate fires (0/k pass, low agreement) ── skips Tier 1 entirely TIER 2 gpt-oss-120b, fresh prompt (no 20b plan), high effort, k = 2-4, same checks ── return best

ALL TIERS – verified-solution store + trace log – LoRA distillation of 20b (Phase 3) – stronger Tier 0 ```

The front door is a cheap classifier, a rule or a small prompt, that sorts tasks by *whether a check exists*, because that determines whether extra compute can help at all. **Tier 0** is a single 20b call at an effort level chosen per family by measurement, followed by an objective check. For code, the check is Neo's sandboxed `run_python` on the task's own examples, plus CodeT-style model-generated tests. For math, it is agreement between the chain-of-thought answer and an independently written program, computed with exact rationals and `solve()`. Passing returns immediately, and on easy traffic this is where the large majority of problems should end.

**Tier 1** spends more 20b compute only where coverage helps: k samples at high effort, selected by execution agreement or answer voting, with k capped near Stroebl's sub-10 optimum. Repair happens once, and only on an objective failure, fed with actual test output. A revision replaces a passing answer only if it passes strictly more tests. A **difficulty gate** sends problems straight to Tier 2 when zero of k samples pass or agreement is low, using the shape of Damani et al.'s adaptive allocation, which cut compute up to 50% at equal quality (arXiv 2410.04707).

**Tier 2** calls 120b with a fresh prompt, never the 20b's failed plan, at high effort with k=2-4 under the same verifier. This is Kapoor's escalation baseline with a better gate. For factual tasks, retrieval runs only when closed-book samples disagree. Queries are formed from the question plus each option, the top passages are reranked and placed at the context edges, and corpora are matched to the domain rather than limited to Wikipedia.

Several existing components change role. **Remove** persona and mood prompts, the planner on single-step tasks, and the LLM checker as a gate. Keep the checker only as a tie-breaker *after* execution, and only if its measured false-negative rate justifies it. **Repurpose** memory into a verified-solution store that holds only solutions which passed checks, together with their tests. That store is both a retrieval source for similar future problems and the training set for Phase 3. This repurposing is a design hypothesis, not a published result.

The cost model makes the trade-off explicit. Let $p_0$ be the fraction verified at Tier 0, $p_1$ the cumulative fraction verified by the end of Tier 1, f the verifier's false-pass rate, and $k_1$ and $k_2$ the sample counts. Then expected cost per problem ≈ $c_{20}\cdot[1 + (1-p_0)\cdot k_1] + (1-p_1)\cdot k_2\cdot c_{120}$. Accuracy ≈ $p_1\cdot(1-f) + (1-p_1)\cdot \mathrm{acc}_{120}|\text{escalated}$. Two
```

consequences follow. First, the verifier's false-pass rate f, not model size, bounds the quality loss, so verifier quality is the system's most important component. Second, because 120b's active compute is only about 1.4x 20b's, the cascade beats raw 120b on cost only if p_0 is high and k_1 stays small. If the user's endpoints price the two models similarly, Claim A's cost advantage will be thin, and the stronger version of the claim becomes "20b system beats raw 120b on accuracy at equal cost". These formulas are derived, not measured; no published study benchmarks a gpt-oss 20b→120b cascade.

Phase 3: the distillation flywheel

Every Tier 2 success produces a verified 120b solution on a problem 20b failed, which is exactly the curated, hard, correct data LIMO and s1 show is most valuable. Periodically LoRA-fine-tune 20b on these traces, a few thousand filtered examples, mixed with real data to avoid collapse. Evaluate only on held-out, post-cutoff problems. If the gain holds, Tier 0's pass rate rises and escalations fall, so the system grows cheaper over time.

On-policy distillation is the more efficient recipe, but it needs teacher log-probabilities on student samples. That requires self-hosting gpt-oss-120b rather than calling an API that may not expose them. The user should also confirm that the terms of any teacher model permit training on its outputs.

The experiments that prove or kill it

#	Experiment	What it decides	Pass criterion
E0	Baseline repair: raw 20b/120b at low/med/high effort, robust answer extraction, n≥500 per benchmark	Whether Neo's raw numbers match the model card; the true reference line	Raw MMLU within ~2 pts of card values
E1	Verifier calibration: label 200+ code/math outputs; measure false-pass and false-fail rates of the LLM checker, example tests, generated tests, PAL cross-check	Which checks are allowed to gate decisions	Gate only on checks with measured f and false-fail rates low enough to keep the cascade net-positive
E2	Headroom census: per-problem pass@1 and pass@k (k=8) for 20b and 120b on the target set	Oracle ceiling of any cascade; size of the "120b-only" slice	Proceed only if the 20b-fail/120b-pass slice and 20b pass@k−pass@1 gap are non-trivial
E3	Ablation of current Neo: remove persona, planner, self-refine and LLM checker one at a time on the coding set; log correct→wrong flips	Confirms the self-talk diagnosis	Flips concentrate in refine/checker stages
E4	Cascade Pareto study vs raw 20b (each effort), raw 120b-high, 120b best-of-k with tests, and Kapoor-style escalation	Claim A	Paired McNemar non-inferiority at pre-registered margin; lower $/solved; on the Pareto frontier
E5	Adaptive budget: difficulty gate vs fixed k	Whether per-query allocation saves compute	≥30% cost reduction at equal accuracy
E6	Gated retrieval on SimpleQA/NQ: closed-book vs always-on vs agreement-gated RAG, with domain corpora	Claim B; ClashEval-style damage	Gated RAG beats 120b closed-book; fewer correct→wrong flips than always-on
E7	Flywheel: LoRA-distill 20b on verified Tier-2 traces; evaluate on held-out post-cutoff problems	Whether the system compounds	Tier-0 pass rate rises; escalation rate falls; no regression elsewhere

Methodology matters as much as architecture. Every configuration should be reported as a tuple of accuracy, mean tokens and dollars, mean calls and p95 latency, then plotted against the baselines, following Kapoor and HAL (arXiv 2407.01502). Comparisons on the same problems need paired tests and several seeds. At 80% accuracy the standard error is about ±4 points at n=100 and ±1.8 at n=500, so 30- and 40-problem samples cannot support any claim of a few points. Test sets must postdate gpt-oss's August 2025 release; LiveCodeBench's rolling problems exist for exactly this purpose (arXiv 2403.07974). Logs should be inspected for shortcuts, since HAL caught agents searching for benchmark answers. The benchmark the user is running now is best read through this lens. Its value depends less on Neo's headline score than on whether the output

records per-problem flips, tokens and the 20b/120b headroom that E2 needs.

Conclusion

The research reframes the goal. "Smaller and smarter" does not come from a cleverer prompt pipeline around a fixed model. It comes from moving each kind of difficulty to the cheapest component that can *certify* it: facts to an index, arithmetic and execution to tools, correctness to tests and checkers. Model compute is spent only where those certificates show it is needed. The model's own judgment about its work is the one resource that consistently fails to scale. That is why debate, critique and refinement loops underperform simple sampling plus an external check, and why Neo's most elaborate features are its least valuable.

Two implications are less obvious. First, gpt-oss-20b and 120b are closer in per-token compute (about 1.4x) than their names suggest. Neo's biggest "smaller" wins will therefore come from verification multiplying 20b's coverage, and from comparing 120b-plus-system against frontier models, more than from routing between the two siblings. Second, the densing trend means the base model under Neo will keep improving faster than any harness. The durable asset is the *verifier stack and the verified-solution data*, which carry over to whichever small model is densest next quarter. That asset is also what can turn a measured, family-specific parity claim into something publishable.